

CloudFormation: IaC nativo en AWS

▼ CloudFormation: IaC nativo en AWS

▼ Framework CI/CD

- Integración Continua (CI)
- Entrega Continua (Continuous Delivery)
- Despliegue Continuo (Continuous Deployment)
- Entrega vs Despliegue Continuo: Diferencia
- Beneficios de CI/CD
- Pipeline CI/CD: Flujo típico
- Herramientas populares de CI/CD

▼ Estrategias de despliegue

- Relación con IaC
- Automatización de infraestructura en pipelines
- Despliegue automatizado de aplicaciones completas
- Empaquetado y artefactos de despliegue
- Entornos de despliegue: dev, test, prod
- Separación de entornos (cuentas y recursos)
- Configuración específica por entorno

▼ Estrategias de despliegue sin downtime

- Despliegue Rolling Update
- Despliegue Blue/Green (entornos paralelos)
- Despliegue Canary

▼ Herramientas para CI/CD Pipelines

▼ AWS CodeDeploy: despliegues automatizados

- Modos de despliegue en CodeDeploy
- CodeDeploy para funciones Lambda
- Rollback automático con CodeDeploy
- Integración de CodeDeploy en CI/CD

▼ ¿Qué es GitHub Actions?

- ¿Cómo funciona GitHub Actions?
- CI/CD con GitHub Actions: flujos típicos
- Ejemplo: Deploy serverless con GitHub Actions
- Autenticación de GitHub Actions con AWS

Framework CI/CD

Integración Continua (CI)

La **integración continua (CI)** es una filosofía de desarrollo donde el equipo de desarrollo integra frecuentemente sus cambios de código en un repositorio común de forma automática, acompañados de pruebas para detectar errores pronto. Esto evita que cada desarrollador trabaje aislado hasta el final, reduciendo conflictos al combinar código. Cada integración activa una compilación automática y una batería de tests (unitarios, integración, etc.) para verificar que el nuevo código funciona con el existente.

El objetivo de CI es identificar y corregir fallos lo antes posible en el ciclo de desarrollo, manteniendo el código en un estado desplegable en todo momento. Esta práctica agiliza el desarrollo y asegura que el proyecto siempre tenga una versión reciente y estable lista para probar o liberar.

Entrega Continua (Continuous Delivery)

La **entrega continua** es la extensión natural de la CI. Consiste en automatizar el proceso de preparación de una versión de software lista para producción de forma fiable. **Después de la etapa de integración y pruebas, la entrega continua garantiza que los cambios aprobados estén empaquetados y listos para desplegar.**

En la entrega continua, a diferencia del despliegue continuo, suele haber una **intervención humana** antes de liberar a producción: el equipo decide cuándo hacer el despliegue final. Es decir, el pipeline automáticamente construye, prueba y deja el artefacto preparado, pero la publicación a los usuarios ocurre bajo control manual.

Esta práctica asegura que siempre haya una build validada disponible para desplegar. Permite lanzamientos frecuentes y confiables, pero con la posibilidad de programarlos en el momento más oportuno para el negocio o realizar revisiones finales antes de impactar al usuario final.

Despliegue Continuo (Continuous Deployment)

El **despliegue continuo** lleva la automatización un paso más allá. En este enfoque, cada cambio que pasa todas las pruebas automatizadas avanza por el pipeline y se **despliega automáticamente en producción**, sin necesidad de aprobación manual. Se elimina completamente la pausa entre integrar cambios y ponerlos en manos de los usuarios.

En otras palabras, **si el código nuevo supera los tests, el sistema CI/CD lo implementa directamente en el entorno productivo**. Esto permite acelerar enormemente el ciclo de desarrollo con entrega final, incorporando nuevas funcionalidades o correcciones a producción en minutos.

La clave es contar con **pruebas automatizadas robustas y monitorización**, ya que no hay revisión manual antes de los usuarios utilicen la nueva versión. El despliegue continuo reduce la sobrecarga operativa y mantiene el producto en evolución constante, con pequeños cambios frecuentes, y esta vez, automatizados.

Entrega vs Despliegue Continuo: Diferencia

Aunque ambos se abrevian "CD", existe una diferencia fundamental: **entrega continua** implica que el sistema prepara automáticamente cada versión candidata pero espera confirmación humana para producir el lanzamiento final, mientras que **despliegue continuo** significa que cada versión validada se lanza automáticamente a producción. En otras palabras, con entrega continua **el equipo aún controla cuándo** desplegar, y con despliegue continuo el sistema despliega inmediatamente tras pasar las pruebas.

Red Hat lo resume así: en la distribución (entrega) continua los cambios **no se implementan automáticamente en producción**, mientras que en la implementación (despliegue) continua **sí**. La elección entre uno u otro depende del grado de excelencia operacional que se desea alcanzar y la tolerancia al riesgo y las necesidades del equipo: organizaciones con alta confianza en su pipeline automático optan por despliegue continuo, mientras otras prefieren un control previo antes de tocar producción.

Beneficios de CI/CD

En primer lugar, **acelera el ciclo de desarrollo** al automatizar tareas manuales: se reducen tiempos de integración, pruebas y despliegue, con versiones más frecuentes. Además, disminuye la cantidad de errores en producción, ya que cada cambio pasa por pruebas rigurosas en el pipeline.

- La automatización continua ayuda a **evitar fallos de integración** y mantiene un flujo constante de actualizaciones de software. También reduce el **tiempo de inactividad** y agiliza los lanzamientos, pues elimina esperas entre fases. Los usuarios finales reciben mejoras más rápido y con mayor regularidad, aumentando su satisfacción.
- Otro beneficio es la **retroalimentación temprana**: al desplegar cambios pequeños y continuos, es más fácil identificar qué introdujo un problema y corregirlo rápidamente. En conjunto, CI/CD mejora la calidad, la eficiencia y la confianza en el proceso de entrega de software.

Pipeline CI/CD: Flujo típico

En un flujo típico de CI/CD, cada cambio de código recorre una **canalización (pipeline)** de etapas automatizadas. Por ejemplo, al hacer *push* de código a la rama principal, un sistema CI inicia la fase de **build** (compilación), donde se construye el proyecto y sus dependencias. Luego ejecuta una suite de **pruebas automatizadas** (unitarias, integración, etc.) para validar el cambio.

Si todo va bien, el pipeline puede proceder a una etapa de **deploy** (despliegue). Dependiendo de la configuración, podría desplegarse automáticamente a un entorno de staging o incluso a producción (si se practica despliegue continuo). En caso de entrega continua, el pipeline dejará listo un artefacto desplegable y esperará aprobación para producción.

Durante todo el proceso, la pipeline provee **feedback inmediato** a los desarrolladores: si falla una compilación o test, se notifica enseguida para corregir. Así, el pipeline CI/CD garantiza que solo los cambios verificados pasen a las siguientes fases y eventualmente a los usuarios finales.

Herramientas populares de CI/CD

Existen numerosas herramientas para implementar pipelines CI/CD. Algunas populares de código abierto o terceros son **Jenkins**, **Travis CI**, **CircleCI** o **GitLab CI**, que permiten definir flujos de integración y despliegue automatizados. También hay servicios en la nube como **GitHub Actions** (integrado en GitHub), y soluciones específicas de proveedores cloud como **AWS CodePipeline** en Amazon Web Services o **Azure DevOps Pipelines**.

Estas plataformas permiten orquestar las distintas etapas (build, test, deploy, etc.) mediante archivos de configuración (por ejemplo, archivos YAML). Muchas incluyen un ecosistema de **plugins o acciones reutilizables** para tareas comunes (como compilar cierto lenguaje, ejecutar tests, desplegar a un servicio específico).

La elección de herramienta dependerá del entorno del proyecto, sinergias con lo existente, etc. Lo importante es que todas cumplen el objetivo de automatizar el ciclo de integración y entrega, dando consistencia y rapidez al proceso de desarrollo.

Estrategias de despliegue

Relación con IaC

Hemos profundizado bastante en la **Infraestructura como Código** es una práctica donde la configuración de servidores, redes, bases de datos y demás recursos se define declarativamente en

archivos de código (generalmente YAML o JSON) en lugar de configurarse manualmente. Esto permite replicar la infraestructura de forma consistente y mantener versiones.

Así, podemos levantar entornos enteros de forma automatizada y repetible y nos aseguramos de que dev, pruebas y prod estén alineados, evitando configuraciones “a mano” inconsistentes. IaC aporta **consistencia entre entornos** (misma configuración en dev/QA/prod) y agiliza escalados o recuperaciones de forma fiable.

Al tratar la infraestructura igual que el código, se pueden aplicar prácticas de CI/CD: cada cambio en “la template” de infraestructura pasa por revisión, pruebas (por ejemplo, validación de la plantilla) y despliegue automatizado, reduciendo errores humanos y tiempo de provisionamiento.

Automatización de infraestructura en pipelines

Integrar IaC en un pipeline permite **desplegar infraestructura automáticamente** cada vez que se actualiza la definición. En lugar de crear recursos manualmente en una consola, el pipeline se encarga de aplicar los cambios de infraestructura descritos en código.

Tal y como vimos, suponiendo que en el repositorio tuviéramos la plantilla de infraestructura (red, servidores, bases de datos). Al hacer commit de una modificación (como añadir una tabla de base de datos), el pipeline puede ejecutar automáticamente comandos para desplegar esos cambios: en AWS, por ejemplo, invocando a CloudFormation para que actualice la infraestructura según la template.

Esto garantiza que la infraestructura esté siempre sincronizada con el código de la aplicación. Los pipelines de infraestructura suelen incluir pasos de **validación** (asegurarse de que la plantilla es correcta), luego **aprovisionamiento automático** de recursos, y en algunos casos pruebas post-despliegue (p. ej., verificar que un servicio respondió en el nuevo servidor), eliminando o minimizando la intervención manual.

Despliegue automatizado de aplicaciones completas

El objetivo final de CI/CD es poder desplegar una **aplicación completa** (tanto el software como la infraestructura que necesita) de forma automatizada y consistente. Esto significa que no solo se actualiza el código de la aplicación, sino que cualquier cambio en bases de datos, colas de mensajes, funciones serverless, etc., también se gestione en el mismo proceso.

En lugar de separar “primero configuro servidores, luego meto el código”, las pipelines modernas pueden manejar ambas cosas. **Por ejemplo, si nuestra aplicación requiere una nueva tabla de base de datos y una nueva versión de la API, el pipeline puede desplegar primero la tabla (vía IaC) y luego el nuevo código que la usa, todo encadenado.**

Este enfoque integral garantiza que no haya **desfase** entre la infraestructura y el código desplegado: la versión de la app siempre corre sobre la infraestructura apropiada. Además, permite **hacer reversión de entornos completos** (rollback total) si algo sale mal, ya que la definición completa del sistema está bajo control. La automatización de despliegue de aplicaciones completas reduce errores (no hay pasos manuales omitidos) y acelera la entrega de nuevas funcionalidades en toda la infraestructura, desde el backend hasta la base de datos.

Empaquetado y artefactos de despliegue

En el proceso de despliegue automatizado, el pipeline genera **artefactos** listos para desplegar. Un artefacto puede ser un archivo compilado (como un **jar** o **binary**), una **imagen de contenedor Docker** o un paquete de funciones. En aplicaciones serverless con SAM, por ejemplo, el comando de build produce un paquete (archivo ZIP con el código de la Lambda) y una versión procesada de la template con referencias a ese código empaquetado.

Estos artefactos son almacenados (por ejemplo, en **Amazon S3** para código Lambda, o en un registro de contenedores para imágenes Docker) de manera que la etapa de despliegue los consuma. El pipeline debe trasladar los artefactos desde la etapa de build a la de deploy; muchas herramientas CI/CD manejan eso automáticamente (subiendo el paquete a un bucket o pasando la imagen al orquestador).

Un beneficio de empaquetar artefactos es la **reproducibilidad**: se despliega exactamente lo que se construyó y probó. Además, permite separar el esfuerzo de compilación (que puede usar entornos especiales, dependencias, etc.) del esfuerzo de despliegue (que solo toma algo ya listo y lo coloca en producción). Al final, el artefacto es la unidad que viaja por el pipeline hacia los diferentes entornos.

Entornos de despliegue: dev, test, prod

En un flujo de CI/CD típico se emplean **múltiples entornos** para separar fases de validación. Comúnmente tenemos al menos: un entorno de **desarrollo (dev)** o integración, donde los desarrolladores despliegan las versiones en desarrollo; un entorno de **pruebas o staging** (QA/test/preproducción) donde se valida la versión candidata en condiciones similares a producción; y finalmente el entorno de **producción (prod)** que utilizan los usuarios finales.

Esta separación permite probar cambios en un ambiente controlado antes de que impacten al cliente real. **Por ejemplo, podemos desplegar automáticamente cada commit al entorno de dev para pruebas iniciales, luego promover una versión estable a staging para realizar pruebas integrales o de usuario, y solo tras pasar esas validaciones, desplegar a producción.**

Cada entorno suele tener **datos y configuraciones aisladas**. Esto garantiza que si algo falla en dev o staging, no afectará al sistema productivo ni a los usuarios reales. Además, facilita iterar rápidamente en dev sin arriesgar la estabilidad del servicio en producción.

Separación de entornos (cuentas y recursos)

Es fundamental **aislar los entornos** entre sí. En AWS, una práctica común es usar **cuentas diferentes** para dev, test y prod, de modo que los recursos (y permisos) estén totalmente separados. Por ejemplo, la base de datos de pruebas estará en la cuenta de staging, mientras la de prod en otra cuenta; así se evita que un error en pruebas borre datos de producción por accidente.

Cuando no se usan cuentas distintas, al menos se separan por proyecto o por nombres de recurso (por ejemplo, prefijos “-dev” y “-prod” en los nombres), o mediante distintos *workspaces* en Terraform. También se pueden usar diferentes **regiones** o **VPCs** para aislamiento adicional.

Además de la división lógica, se suele asignar diferentes credenciales de acceso: los desarrolladores pueden tener permisos más amplios en dev, pero muy restringidos en prod. Las pipelines CI/CD manejan credenciales separadas para desplegar en cada entorno, garantizando seguridad y control.

En resumen, separar entornos proporciona una red de seguridad: cada uno es un sandbox independiente para pruebas, sin interferir con el entorno de nivel superior. Esto mejora la confiabilidad de los despliegues graduales hacia producción.

Configuración específica por entorno

Aunque buscamos paridad entre entornos, hay parámetros que necesariamente difieren entre **dev, test y prod**. Por ejemplo, cada entorno suele tener su propia base de datos, credenciales API distintas, tamaños de servidor ajustados a la carga, etc. La pipeline debe gestionar estas diferencias de configuración sin necesidad de alterar el código fuente para cada despliegue.

Una técnica común es el uso de **archivos de configuración o parámetros por entorno**. En CloudFormation/SAM, por ejemplo, podemos pasar distintos parámetros o utilizar distinta *Parameter Store*/**Secrets Manager** según el destino. También GitHub Actions o Jenkins permiten definir variables de entorno específicas para cada etapa.

Esto asegura que la aplicación desplegada en “staging” apunta a la base de datos de staging, y la de prod a la de prod, sin mezclas. Asimismo, habilita probar con datos ficticios en QA mientras en prod están los datos reales. La clave está en **no hardcodear datos sensibles o específicos en el código**, sino referenciarlos externamente. El pipeline se encarga de inyectar los valores correctos en

cada entorno. Así, el mismo paquete de aplicación puede desplegarse en todos lados, recibiendo la configuración adecuada en cada uno.

Estrategias de despliegue sin downtime

Para **minimizar interrupciones** al usuario durante un despliegue e introducir cambios de forma segura, existen varias **estrategias de despliegue** avanzadas. Las más comunes son:

- **Rolling Update:** se actualizan gradualmente las instancias o contenedores de la aplicación, un conjunto a la vez, hasta sustituir toda la infraestructura vieja por la nueva versión.
- **Blue/Green:** se mantienen dos entornos en paralelo (azul y verde). El azul sirve la versión actual, mientras en el verde se despliega la nueva versión. Luego se conmuta el tráfico del azul al verde una vez lista la nueva versión. Permite volver atrás simplemente regresando el tráfico al azul si hay problemas.
- **Canary Release:** es similar al blue/green pero a nivel de tráfico: se libera la nueva versión solo a un pequeño porcentaje de usuarios inicialmente (canarios), manteniendo el resto con la versión antigua. Si todo va bien, se incrementa gradualmente el porcentaje hacia la nueva versión.

Estas estrategias son consideradas **buenas prácticas DevOps** para lograr despliegues con alta disponibilidad y menor riesgo. Cada una tiene sus pros y contras en complejidad y uso de recursos, pero todas evitan el enfoque tradicional de “corte directo” (all-at-once) que suele implicar downtime.

Despliegue Rolling Update

En un **despliegue rolling**, la nueva versión de la aplicación se va desplegando por partes sobre la infraestructura existente. Por ejemplo, en un cluster de 10 servidores, se podría actualizar de 2 en 2: se quitan 2 servidores del balanceador, se actualizan a la nueva versión y se vuelven a ubicar. Se repite por tandas determinadas hasta haber reemplazado todos. Durante el proceso, siempre hay instancias atendiendo con la versión anterior mientras otras se actualizan, evitando una caída total del servicio.

La ventaja es que **no requiere duplicar todo el entorno** (se reutiliza la misma infraestructura). Además, suele completarse más rápido que un Blue/Green completo, ya que no hace falta preparar un entorno paralelo completo. Sin embargo, tiene sus desafíos: al no haber un entorno aislado nuevo, cualquier problema en la nueva versión puede afectar a una porción del tráfico mientras dura el despliegue. El rollback es más complicado: implicaría desplegar nuevamente la versión anterior sobre las instancias ya actualizadas, lo que puede ser lento.

Aun así, muchas plataformas (como Kubernetes con sus rolling updates) gestionan esto automáticamente, pausando el despliegue si detectan fallos. Un rolling update bien configurado puede

lograr cero downtime, pero hay que monitorizar con detalle durante la transición.

Despliegue Blue/Green (entornos paralelos)

La estrategia **Blue/Green** consiste en tener dos entornos de producción idénticos: el entorno *Blue* (azul) con la versión actual y estable de la aplicación, y el entorno *Green* (verde) con la nueva versión preparada. Inicialmente, todo el tráfico de usuarios va al entorno azul. La nueva versión se despliega en el verde sin afectar a los usuarios, y realizamos pruebas y verificaciones exhaustivas ahí.

Cuando la versión verde ha sido validada, se realiza el **switch de tráfico**: los usuarios pasan a ser atendidos por el entorno verde (a veces instantáneamente cambiando un alias o balanceador de carga, otras de forma gradual). En ese momento la nueva versión queda activa en producción. Si algo falla tras el cambio, la **reversión** es trivial: se redirige de nuevo el tráfico al entorno azul, que aún conserva la versión previa. Esto permite un rollback casi instantáneo.

Blue/Green ofrece **cero downtime** perceptible y una altísima seguridad al revertir, a costa de recursos duplicados temporalmente (mantener dos entornos completos). Es ideal cuando se requiere máxima confiabilidad en despliegues o actualizaciones de base de datos esquemas, etc, ya que se puede probar todo en el entorno verde antes de exponerlo. Muchas herramientas cloud (como AWS Elastic Beanstalk, CloudFormation, etc.) soportan este patrón nativamente.

Despliegue Canary

El **despliegue canary** es un enfoque de actualización progresiva centrado en el porcentaje de usuarios. En vez de cambiar todo el entorno o actualizar todas las instancias de golpe, se introduce la nueva versión a un **subconjunto pequeño de usuarios** inicialmente. Por ejemplo, se puede dirigir un 5% del tráfico (usuarios “canarios”) a la nueva versión, mientras el 95% restante sigue con la versión antigua.

Durante esta fase, se **monitoriza cuidadosamente** el comportamiento de la nueva versión: métricas de errores, rendimiento, feedback de esos usuarios iniciales. Si no surgen problemas, se incrementa gradualmente el porcentaje de usuarios que reciben la nueva versión (por ejemplo, 20%, luego 50%, etc.) hasta alcanzar el 100%. En caso de detectarse algún fallo grave, el despliegue se detiene y se regresa el tráfico al 0% nuevo (o sea, rollback completo a la versión previa) antes de que la mayoría de usuarios se vean afectados.

La analogía del “**canario en la mina**” refleja que primero se expone un grupo pequeño para asegurar que es seguro continuar. Esta estrategia minimiza el impacto de bugs desconocidos, ya que solo un pequeño segmento experimentaría el problema inicial.

Herramientas para CI/CD Pipelines

AWS CodeDeploy: despliegues automatizados

AWS CodeDeploy es un servicio gestionado de AWS que automatiza la distribución de nuevas versiones de aplicaciones a diferentes servicios de cómputo (instancias EC2, servidores on-premise, clústeres ECS y funciones Lambda). Su objetivo es simplificar el proceso de despliegue, manejando por nosotros tareas como detener servicios, copiar archivos, ejecutar scripts previos/posteriores, y administrar el routing de tráfico en despliegues avanzados.

CodeDeploy soporta distintos **modos de despliegue** según la plataforma de destino. Por ejemplo, para instancias EC2 (o servidores físicos) permite despliegues en el lugar (in-place) actualizando instancias existentes gradualmente, o despliegues Blue/Green lanzando nuevas instancias y conmutando tráfico. En contenedores (ECS) también facilita Blue/Green creando nuevos task sets. Y para **AWS Lambda**, CodeDeploy realiza despliegues tipo canary/linear utilizando alias de función para distribuir tráfico.

La ventaja de CodeDeploy es que nos da un **marco uniforme** para implementar estrategias como rolling, blue/green o canary sin construir la lógica manualmente. Podemos definir políticas de despliegue (porcentaje, intervalos, condiciones de rollback) y CodeDeploy orquesta los pasos necesarios para lograrlo en la plataforma correspondiente. De esta forma, se reduce el riesgo de error humano en los despliegues y se logra consistencia entre entornos.

Modos de despliegue en CodeDeploy

Según el tipo de destino, CodeDeploy maneja el despliegue de forma algo distinta:

- **EC2/On-Premises:** Puede hacer **despliegue en in-place**, deteniendo cada instancia temporalmente y actualizando la aplicación (lo que equivale a un rolling). O bien puede realizar **Blue/Green**, lanzando nuevas instancias (verde) con la nueva versión a la vez que las antiguas (azul) mediante proceso de registrar/deregistrar instancias en el balanceador.
- **Amazon ECS (contenedores):** CodeDeploy integra con ECS y Application Load Balancer para implementar Blue/Green a nivel de tareas. Crea un nuevo **task set** con la versión nueva de la tarea y redirige tráfico gradualmente del task set antiguo al nuevo. Esto permite actualizar microservicios en ECS con cero downtime.
- **AWS Lambda:** No existen “instancias” dedicadas por actualizar, así que CodeDeploy utiliza la técnica de **alias con ponderación de tráfico**. Básicamente mantiene dos versiones de la función Lambda: la antigua apuntada por un alias (ej. “prod”) y la nueva versión publicada y reconfigura el alias para enviar una fracción de invocaciones a la nueva versión.

Además, CodeDeploy permite especificar *hooks* en distintas fases (antes de instalar, después de instalar...) para personalizar el proceso, aunque en Lambda, en lugar de hooks se suele emplear alarmas de CloudWatch.

CodeDeploy para funciones Lambda

En el contexto de **AWS Lambda**, CodeDeploy implementa los despliegues avanzados usando versiones y alias de las funciones. Cuando subimos una nueva versión de una Lambda, CodeDeploy (configurado vía SAM o manualmente) crea o identifica un alias (por ejemplo “prod” o “live”) que apunta a la versión vigente. Al iniciar un despliegue canary o linear, CodeDeploy ajusta ese alias para que enrute un porcentaje del tráfico a la nueva versión y el resto siga en la versión previa. Por ejemplo, en un canary 10%, durante unos minutos el alias dirige 90% de invocaciones a la versión antigua y 10% a la nueva.

Si no se detectan problemas, CodeDeploy actualiza progresivamente la ponderación del alias hasta apuntarlo 100% a la nueva versión. En un despliegue linear, este incremento ocurre en pasos iguales (ej. +10% cada X minutos). Todo este cambio de tráfico es transparente para el cliente que invoca la función, solo varía qué versión atiende la petición.

Si rascamos un poco la fachada, por debajo CodeDeploy aprovecha que Lambda puede tener muchas versiones activas (inmutables) y un alias que actúa como endpoint fijo. Para los desarrolladores, configurar esto es sencillo mediante SAM: por ejemplo, definiendo propiedades como **AutoPublishAlias** y **DeploymentPreference** en la template, se indica automáticamente a CodeDeploy que use cierto tipo de canary/linear y los umbrales de monitorización deseados.

Rollback automático con CodeDeploy

Una de las características más poderosas de CodeDeploy es la capacidad de **rollback automático** en caso de fallo. Podemos asociar **alarmas de Amazon CloudWatch** a un proceso de despliegue: por ejemplo, una alarma que se active si la nueva versión de la aplicación registra errores en más del 5% de las solicitudes, o si el tiempo de respuesta supera cierto umbral. CodeDeploy monitoriza estas alarmas durante el despliegue gradual.

Si alguna alarma cruza el umbral definido (indicando que la versión nueva está causando problemas), CodeDeploy detiene el despliegue y **revierte automáticamente a la versión anterior estable**. **En la práctica, para Lambdas esto significa reconfigurar el alias de inmediato de vuelta al 100% en la versión antigua**. En un despliegue EC2 blue/green, implicaría volver a enrutar todo el tráfico al grupo azul original. Todo esto sucede sin intervención humana y típicamente en segundos.

Gracias a esto, el **MTTR (Mean Time to Recovery)** mejora dramáticamente: el sistema se auto-restaura antes de que muchos usuarios noten el fallo. Después de un rollback automático, el pipeline

o los ingenieros pueden analizar el problema con calma, sabiendo que producción sigue corriendo con la versión previa. Esta “red de seguridad” automatizada da mucha confianza para desplegar con frecuencia, ya que si algo sale mal, CodeDeploy actúa como paracaídas devolviendo el servicio al estado sano.

Integración de CodeDeploy en CI/CD

CodeDeploy normalmente se integra como una etapa dentro del pipeline CI/CD global. Por ejemplo, en **AWS CodePipeline** se puede añadir una acción de despliegue que apunta a CodeDeploy, de modo que tras la construcción y pruebas, CodePipeline invoca CodeDeploy para desplegar la nueva versión según la estrategia configurada (blue/green, canary, etc.). De igual forma, desde **GitHub Actions** u otras plataformas, se puede llamar a los APIs de CodeDeploy o usar herramientas CLI para iniciar el despliegue.

En aplicaciones serverless con SAM, la integración es aún más sencilla: se define en la plantilla SAM la preferencia de despliegue (canary/linear) y las alarmas, y al hacer `sam deploy`, internamente CloudFormation crea una **App** y **Deployment Group** de CodeDeploy asociados a la Lambda. Así, cada vez que subimos nuevo código con SAM, CodeDeploy orquesta el cambio de alias en la Lambda automáticamente según la política elegida, sin que tengamos que invocar manualmente CodeDeploy.

Es importante también integrar notificaciones: CodeDeploy puede enviar eventos (via SNS, CloudWatch Events/EventBridge) que el pipeline puede escuchar para saber si un despliegue fue exitoso o si hizo rollback. Así, el pipeline podría decidir promover a producción solo si el despliegue en staging tuvo éxito completo sin rollback, por ejemplo. Esta sinergia entre pipeline y CodeDeploy permite un **flujo CI/CD robusto** que abarca desde el commit hasta el manejo seguro en producción.

¿Qué es GitHub Actions?

GitHub Actions es la plataforma de CI/CD integrada en GitHub. Permite automatizar tareas en respuesta a eventos del repositorio, mediante flujos de trabajo definibles por el usuario. En otras palabras, con Actions puedes configurar pipelines (llamados *workflows*) que se ejecutan cuando ocurre algo en tu repo: por ejemplo, un push, una pull request, la creación de un tag, etc.

Es una solución nativa de GitHub, lo que significa que no necesitas montar un servidor de CI aparte (como con Jenkins u otros). GitHub provee la infraestructura para ejecutar los jobs. Con Actions puedes compilar código, correr tests, construir imágenes de Docker, publicar paquetes, desplegar a servidores en la nube, y prácticamente cualquier cosa que puedas poner en un script.

Un aspecto potente es que existen numerosas **acciones reutilizables** en el Marketplace de GitHub (publicadas por la comunidad o por proveedores) que simplifican las tareas comunes. Por ejemplo,

hay acciones para configurar un entorno con cierto lenguaje, acciones para desplegar en AWS, Google Cloud, etc. Puedes usarlas como bloques de construcción en tus flujos de trabajo en vez de escribir todo desde cero. Además, GitHub Actions es gratuito para repositorios públicos y ofrece minutos gratuitos mensuales para repos privados, lo cual lo hace muy accesible.

¿Cómo funciona GitHub Actions?

GitHub Actions funciona mediante la definición de **workflows** (flujos de trabajo) en archivos YAML dentro del repositorio (normalmente en `.github/workflows/`). Cada workflow especifica en qué **eventos** debe activarse (por ejemplo, *on: push* en la rama main, o *on: pull_request*). Cuando ocurre ese evento, GitHub inicia el workflow.

Un workflow consta de uno o varios **jobs**. Cada job es una serie de pasos que se ejecutan en un **runner** (una máquina virtual o contenedor proporcionado por GitHub, disponible con sistemas Linux, Windows, macOS). Los jobs de un workflow pueden ejecutarse en paralelo o secuencialmente según dependencias que definamos. Dentro de cada job, hay **steps** (pasos) que pueden ser ejecutar una **acción** reutilizable o un simple comando de shell.

Por ejemplo, un job típico puede tener pasos: usar una acción oficial de checkout para obtener el código del repo, luego un paso de set-up (instalar dependencias), luego un paso de build (ejecutar `npm build` por decir algo), después un paso de test (`npm test`), y finalmente un paso de deploy (quizá usando una acción que interactúe con un cloud). Cada step se ejecuta en orden dentro del job, compartiendo el mismo runner (y pueden compartir archivos generados).

Los **eventos** que activan workflows abarcan casi cualquier actividad en GitHub: push, pull request, creación de release, issues comentados, etc. Incluso se pueden programar ejecuciones (cron) o disparar manualmente.

CI/CD con GitHub Actions: flujos típicos

Usar GitHub Actions para CI/CD es muy práctico. Un flujo típico podría ser: tienes un workflow configurado *on: push* a la rama principal. Cuando subes código, se dispara el workflow de **Integración Continua**: un job de build compila la aplicación y ejecuta los tests automáticamente. Si algún test falla, la ejecución marca error (y GitHub puede avisarte vía email o en la interfaz, incluso impedir merge si así lo configuras).

Si todo pasa, el mismo u otro workflow puede encargarse de la **Entrega/Despliegue Continuo**. Por ejemplo, podrías tener un workflow *on: push tags* (al crear un tag de versión) que construya la imagen Docker de tu app y la publique a un registro, y luego despliegue esa versión a un servidor o a la nube. O un workflow que al hacer merge a `main` despliegue automáticamente a un entorno de staging.

Incluso **es posible combinarlo con aprobaciones manuales usando GitHub Environments** (que permiten requerir aprobaciones para usar ciertos secretos de despliegue, simulando un gate).

Un punto fuerte es la capacidad de usar **acciones preexistentes**. Por ejemplo, GitHub ofrece la acción `actions/checkout@v3` para clonar el repo, o `actions/setup-node` para preparar Node.js. AWS ofrece acciones para configurar credenciales y hasta desplegar CloudFormation. Así, en tu pipeline solo “armas el lego”: checkout código, set up lenguaje, build, test, desplegar con acción X. Esto acelera mucho escribir la configuración.

Ejemplo: Deploy serverless con GitHub Actions

Imaginemos que queremos implementar nuestro caso práctico (la app Lambda+API Gateway+DynamoDB) con GitHub Actions en lugar de CodePipeline. Podríamos crear un workflow YAML con un pipeline así:

- **Trigger:** on push a la rama `main` (o cuando hagamos una release).
- **Build job:** Checkout del repositorio, configurar AWS SAM CLI (usando una acción oficial que la instala), luego ejecutar `sam build` para compilar la Lambda y preparar el paquete. También ejecutaríamos los tests locales (p. ej. usando `pytest` si es Python, o pruebas de integración con SAM Local).
- **Deploy to dev job:** Solo si pasó el build. Usar la acción **configure-aws-credentials** de AWS para asumir un rol de despliegue en la cuenta de desarrollo (utilizando OpenID Connect, sin manejar claves directamente). Después, ejecutar `sam deploy` apuntando al entorno dev (stack de CloudFormation dev), quizás con la opción `--no-confirm-changeset` para que sea no interactivo. Esto creará/actualizará recursos en dev y desplegará la nueva versión de la Lambda allí.
- (Opcional) **Test en dev job:** Se podrían invocar funciones de prueba o endpoints en dev para verificar que todo esté OK tras el despliegue.
- **Promoción a prod (manual):** Podríamos usar **GitHub Environments** para producción, de modo que tras el job de dev, quede pendiente una aprobación manual. Al aprobar, se ejecuta el job de deploy a prod. Éste haría otra vez `configure-aws-credentials` (ahora con rol en la cuenta prod) y `sam deploy` hacia el stack de producción. La template SAM incluye la configuración de CodeDeploy (canary con alarmas), así que este `sam deploy` automáticamente iniciará el despliegue controlado en prod (10% y rollback automático si hay alarmas).
- **Notificaciones:** Finalmente, podríamos integrar un paso que notifique en un canal (email/Slack) que la versión fue desplegada con éxito, o que hubo un rollback si se detectó un problema (esto podría hacerse leyendo el resultado de `sam deploy` o eventos de CodeDeploy).

Este ejemplo muestra cómo Actions puede orquestar todo: construye, prueba y despliega en dos entornos usando SAM y CodeDeploy detrás. Todo definido como código en el repositorio. Así

obtenemos un flujo CI/CD completo utilizando GitHub Actions para implementar nuestra aplicación serverless de forma segura y automatizada.

Autenticación de GitHub Actions con AWS

Para que GitHub Actions pueda desplegar recursos en AWS (como en el ejemplo anterior), es necesario configurar las credenciales de AWS en el workflow de forma segura. Existen dos métodos principales:

1. **Access Key/Secret:** crear un usuario IAM con permisos limitados y almacenar sus claves (ID y Secret) como *secrets* en GitHub. Luego, en el workflow, usar esas claves para configurar las credenciales (`AWS_ACCESS_KEY_ID` y `AWS_SECRET_ACCESS_KEY`) antes de ejecutar comandos AWS. Este método funciona pero tiene riesgo de manejo de secretos estáticos. Recordemos que en **AWS Academy** es necesario incluir también `AWS_SESSION_TOKEN` .
2. **OpenID Connect (OIDC):** es la alternativa más moderna y segura. GitHub Actions puede autenticarse contra AWS mediante OIDC, de modo que no se necesita almacenar claves largas. Se configura un **proveedor de identidad OIDC** en AWS que confía en GitHub. Luego se crea un rol IAM que la Action asume temporalmente. GitHub provee la acción `aws-actions/configure-aws-credentials` que simplifica esto: uno especifica el rol a asumir y la región, y la Action realiza el intercambio de tokens por nosotros. Recordemos también que **esta opción no está disponible en laboratorios de AWS Academy**.

En ambos casos, una vez configuradas las credenciales en el runner, se pueden usar las herramientas de AWS (CLI, SAM, etc.) libremente para desplegar. Así, GitHub Actions puede integrarse con AWS de forma controlada y segura, habilitando despliegues automatizados sin exponer credenciales sensibles públicamente.